IntraNet

Dashboard de Comunicação Interna

PRD — Product Requirements Document

Alunos:
João David Salomão
Lucas da Silva Santos
Pollyana de Castro Rodrigues

Disciplina: Modelagem de Dados | Ano: 2026

0. Objetivo do Produto

O IntraNet tem como objetivo centralizar postagens, perfis de colaboradores e comentários em uma interface web única, substituindo o uso informal de e-mail e grupos de WhatsApp para comunicação interna da equipe. O protótipo valida a viabilidade técnica da solução consumindo a API pública JSONPlaceholder, eliminando a necessidade de back-end e banco de dados próprios na fase inicial, acelerando a entrega de valor e reduzindo custos de desenvolvimento.

O produto responde à demanda da diretoria por uma solução de comunicação interna moderna, estruturada e rastreável, que permita a qualquer funcionário ou gestor acessar postagens, consultar perfis e acompanhar discussões em um único ponto de acesso.

1. Visão Geral do Produto

Nome	IntraNet
Tipo	Protótipo web — Single Page Application (SPA)
Arquitetura	Cliente-Servidor · Padrão MVC · HTTP REST
Fonte de Dados	JSONPlaceholder (API pública, formato JSON)
Usuários-alvo	Funcionários e gestores da startup
Plataforma	Navegador web moderno — sem instalação necessária
Repositório	GitHub (código-fonte público da equipe)

O IntraNet é um protótipo funcional que demonstra como uma arquitetura baseada em serviços RESTful pode sustentar um produto interno de comunicação sem a necessidade de infraestrutura back-end dedicada na fase de validação. A interface consome dados reais em formato JSON e aplica as camadas Model, View e Controller para organizar a lógica de negócio e a apresentação.

2. Escopo do Projeto

2.1 Dentro do Escopo

- Listagem de todos os colaboradores retornados pelo endpoint GET /users
- Exibição de postagens de um colaborador selecionado via GET /posts?userId={id}
- Carregamento de comentários de uma postagem via GET /comments?postId={id}
- Busca e filtragem local de colaboradores por nome ou e-mail (sem nova requisição)
- Tratamento de falhas de rede: timeout via AbortController, try/catch, mensagens de erro amigáveis
- Cache em memória de comentários já carregados para evitar requisições duplicadas
- Estados de carregamento (skeleton) durante requisições pendentes
- Opção de nova tentativa em caso de falha de requisição
- Compatibilidade com navegadores modernos: Chrome, Firefox, Safari e Edge

2.2 Fora do Escopo

- Autenticação e controle de acesso (login/logout, perfis de permissão)
- Criação, edição ou exclusão de dados — verbos POST, PUT e DELETE não são utilizados

- Back-end próprio, banco de dados relacional ou NoSQL
- Notificações em tempo real (WebSocket, Server-Sent Events)
- Responsividade mobile avançada ou aplicativo nativo
- Testes automatizados — unitários, de integração ou end-to-end
- Integração com sistemas terceiros além do JSONPlaceholder
- Internacionalização (i18n) ou suporte a múltiplos idiomas

3. Modelo de Dados

As entidades do sistema são mapeadas diretamente a partir da estrutura JSON retornada pelos endpoints da API JSONPlaceholder. Abaixo estão as três classes principais do domínio do cliente e seus atributos, conforme utilizados no protótipo.

3.1 Classe Usuario

Atributo	Tipo	Fonte (JSON)
id	int	GET /users
name	string	GET /users
username	string	GET /users
email	string	GET /users
phone	string	GET /users
website	string	GET /users

3.2 Classe Postagem

Atributo	Tipo	Fonte (JSON)
id	int	GET /posts
userId	int (FK)	GET /posts
title	string	GET /posts?userId={id}
body	string	GET /posts?userId={id}

3.3 Classe Comentario

Atributo	Tipo	Fonte (JSON)
id	int	GET /comments
postId	int (FK)	GET /comments?postId={id}
name	string	GET /comments?postId={id}
email	string	GET /comments?postId={id}
body	string	GET /comments?postId={id}

3.4 Associações entre Entidades

As relações entre as classes seguem a estrutura dos dados retornados pela API:

- Usuario 1 ↔ * Postagem: um colaborador pode ter muitas postagens. A chave estrangeira userId em Postagem referencia o id de Usuario.
- Postagem 1 ↔ * Comentario: uma postagem pode ter muitos comentários. A chave estrangeira postId em Comentario referencia o id de Postagem.

Essas associações são representadas no Diagrama de Classes UML entregue separadamente pela equipe, com multiplicidade 1:* para ambas as relações.

4. Arquitetura — Padrão MVC

O código do protótipo é organizado segundo o padrão Model-View-Controller (MVC), separando claramente as responsabilidades de cada camada e facilitando manutenção e evolução do sistema.

Camada	Responsabilidade	Exemplos no IntraNet
Model	Requisições HTTP à API, AbortController, try/catch, cache em memória	fetchUsers(), fetchPosts(), fetchComments(), cache de comentários
View	Renderização do HTML, estados de loading (skeleton), mensagens de erro	Cards de usuário, lista de postagens, modal de comentários, toast de erro
Controller	Ligação entre eventos do usuário e chamadas ao Model; atualização da View	Click em colaborador, acionamento do filtro de busca, botão 'Ver comentários'

A separação em camadas garante que alterações na API (ex: mudança de formato JSON) sejam absorvidas apenas no Model, sem impacto direto na View — reduzindo o risco de integração identificado no Gerenciamento de Riscos do pitch.

5. User Stories (Histórias de Usuário)

As histórias de usuário descrevem as funcionalidades do sistema do ponto de vista de quem o utiliza, seguindo o formato: Como [perfil], desejo [ação], para que [benefício].

ID	Perfil	Desejo	Benefício	Endpoint REST
US01	Funcionário	Ver lista de colaboradores com foto, nome e e-mail	Encontrar contatos rapidamente	GET /users
US02	Funcionário	Ver postagens recentes de um colaborador específico	Manter-se atualizado sobre projetos e ideias da equipe	GET /posts?userId={id}
US03	Gestor	Visualizar comentários de uma postagem	Acompanhar o engajamento e discussões da equipe	GET /comments?postId={id}
US04	Funcionário	Buscar colaborador por nome ou e-mail	Acessar o perfil de um colega sem rolar a lista inteira	Filtro local
US05	Usuário	Receber alerta visual quando a API estiver indisponível ou lenta	Saber quando o sistema está com problemas sem ficar aguardando sem feedback	AbortController + Toast

6. Requisitos Funcionais

Os requisitos funcionais descrevem o que o sistema deve fazer — as funcionalidades concretas implementadas no protótipo.

RF-01 — O sistema deve listar todos os colaboradores retornados por GET /users.

RF-02 — Ao selecionar um colaborador, o sistema deve carregar suas postagens via GET /posts?userId={id}.

RF-03 — Ao clicar em "Ver comentários", o sistema deve carregar os comentários via GET /comments?postId={id}.

RF-04 — O sistema deve exibir um campo de busca que filtre colaboradores por nome ou e-mail em tempo real.

RF-05 — O sistema deve exibir estados de carregamento (skeleton) durante requisições pendentes.

RF-06 — O sistema deve exibir mensagens de erro quando requisições falharem, com opção de nova tentativa.

RF-07 — O sistema deve cachear comentários já carregados para evitar requisições duplicadas.

7. Requisitos Não-Funcionais

Os requisitos não-funcionais definem como o sistema deve se comportar em termos de qualidade, desempenho e resiliência — especialmente em relação à comunicação com a API externa.

RNF-01 — Timeout: Toda requisição à API deve ser cancelada automaticamente após 8 segundos sem resposta, usando AbortController.

RNF-02 — Tratamento de exceções: Toda chamada de rede deve estar envolvida em bloco try/catch. Erros HTTP (404, 500, etc.) devem ser capturados e exibidos ao usuário de forma amigável.

RNF-03 — Disponibilidade percebida: O sistema nunca deve travar. Em caso de falha de rede, o usuário deve sempre receber feedback e uma opção de nova tentativa.

RNF-04 — Performance: O filtro de usuários deve operar sobre dados já carregados em memória, sem novas requisições, garantindo resposta imediata.

RNF-05 — Compatibilidade: O protótipo deve funcionar nos navegadores modernos (Chrome, Firefox, Safari, Edge) sem necessidade de instalação ou build

8. Prompt Utilizado para Geração das User Stories

Conforme solicitado, registra-se abaixo o prompt utilizado no Claude para auxiliar na elaboração das histórias de usuário deste documento:

> *"Estou desenvolvendo um protótipo de um Dashboard de Comunicação Interna para uma startup, como trabalho da disciplina de Modelagem de Dados. O sistema consome dados da API pública JSONPlaceholder, que disponibiliza os endpoints GET /users (lista de colaboradores), GET /posts?userId={id} (postagens por colaborador) e GET /comments?postId={id} (comentários por postagem). O produto se chama IntraNet e tem como objetivo centralizar postagens, perfis de colaboradores e comentários em uma interface web moderna, substituindo o uso informal de e-mails e grupos de WhatsApp para comunicação interna. Com base nesse contexto, gere 5 User Stories no formato ágil padrão: 'Como [perfil do usuário], desejo [ação ou funcionalidade], para que [benefício ou objetivo].' Os perfis de usuário possíveis são: Funcionário, Gestor e Usuário genérico. Cada história deve estar diretamente relacionada a uma funcionalidade real que o sistema entregará, e ao menos uma delas deve tratar de tolerância a falhas de rede ou indisponibilidade da API. Gere Também o Documento PRD completo contendo as especificações necessárias para o Projeto ser totalmente viável."*